

Plan de Branching — Git Flow

Proyecto Sanos y Salvos — Evaluación Parcial N°2

DSY1106 Desarrollo Fullstack III

Integrante	Rol
Benjamin Arellano Gallardo	Desarrollador Fullstack
Benjamin Valdebenito	Desarrollador Fullstack
Maximiliano Vera	Desarrollador Fullstack
Matias Guzman	Desarrollador Fullstack

1. Introducción

El presente documento describe la estrategia de branching adoptada para el desarrollo del proyecto Sanos y Salvos. Se utilizó Git Flow adaptado con ramas nombradas según el componente implementado, garantizando trazabilidad y colaboración ordenada.

2. Estructura de Ramas

2.1 Ramas principales

main: Rama de producción estable. Recibe merges desde develop solo cuando el proyecto está en estado entregable.

develop: Rama de integración continua. Todas las feature branches hacen merge aquí al completarse.

2.2 Ramas de funcionalidad

Rama	Descripción	Merge hacia
feature/msvc-mascotas	Microservicio mascotas — Builder + MySQL	develop
feature/msvc-reportes	Microservicio reportes — Facade + MySQL	develop
feature/msvc-usuarios	Microservicio usuarios — Adapter + MySQL	develop
feature/bff	Backend For Frontend — Facade	develop
feature/infrastructure	API Gateway, Eureka, Config Server, Admin Server	develop
feature/frontend	React + Vite + Keycloak JS + Axios	develop
feature/arquetipos	Arquetipos Maven microservicio y BFF	develop
feature/docs	Documentación PDF del proyecto	develop
feature/docker-keycloak	Docker Compose: Keycloak + MySQL	develop

3. Flujo de Trabajo

Flujo estándar aplicado en cada componente:

1. Crear la rama desde develop:

```
git checkout develop
git checkout -b feature/nombre-componente
```

2. Implementar el componente y hacer commit:

```
git add .
git commit -m "feat: descripcion del componente"
```

3. Push y merge a develop sin borrar la rama (evidencia):

```
git push -u origin feature/nombre-componente
git checkout develop
git merge feature/nombre-componente
git push origin develop
```

4. Gestión de Conflictos

4.1 Convención de nombres de ramas

Conflicto inicial: nombrar ramas por integrante (feature/benjamin) vs por componente (feature/msvc-mascotas). El primer enfoque dificultaba identificar qué trabajo contenía cada rama sin revisar el código.

Resolución: Se acordó por consenso usar feature/nombre-componente, logrando trazabilidad inmediata entre rama y funcionalidad.

4.2 Push rechazado en main (non-fast-forward)

Al crear el repositorio en GitHub, se generó un commit inicial automático que no existía en el repositorio local, causando el error non-fast-forward al hacer push.

Resolución: Se utilizó `git push origin main --force` en la etapa inicial del proyecto, antes de tener trabajo relevante en el repositorio remoto. Para futuros pushes se acordó siempre usar `git pull` antes de push.

4.3 Conflicto de puertos en servicios de infraestructura

Durante el desarrollo se detectó que admin-server y keycloak-adapter estaban configurados en el mismo puerto (9090), que además coincidía con el puerto de Keycloak levantado con Docker.

Resolución: Se redistribuyeron los puertos: Keycloak→9090, admin-server→8085, keycloak-adapter→8086, garantizando que ningún servicio compita por el mismo puerto.

4.4 Driver MySQL no encontrado

Al migrar de H2 a MySQL, los microservicios fallaban con Cannot load driver class: com.mysql.cj.jdbc.Driver. La dependencia estaba en el pom.xml pero Maven no la había descargado.

Resolución: Se ejecutó Reload All Maven Projects en IntelliJ y mvn dependency:resolve -U para forzar la descarga del conector MySQL.

5. Evidencia de Merges

Todas las ramas feature se conservan sin eliminar en el repositorio remoto como evidencia del desarrollo incremental. Los merges están documentados en el historial de GitHub. Repositorio:

<https://github.com/MafeXX/sanos-y-salvos-platform>